
OpenCV

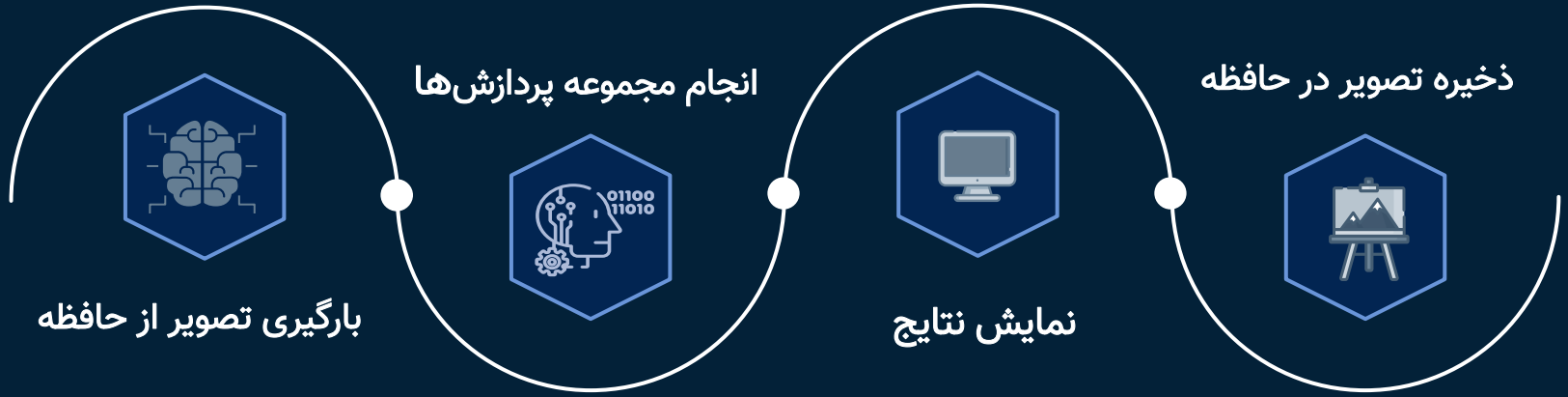




استاد : دکتر ثابتی

ارائه دهنده : مبینا زنگوئی

مراحل اصلی یک برنامه پردازش تصویر




```python
# بارگذاری تصویر
image = cv2.imread('opencv.jpg')
```

```python
# ذخیره و نمایش تصویر
cv2.imshow('Image', image)
cv2.imwrite('resized_image.jpg', resized_image)
```

تبدیل به مقیاس خاکستری

opencv.py > ...

```python
import cv2

# خواندن تصویر
image = cv2.imread('opencv.jpg')

# تبدیل تصویر به مقیاس خاکستری
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# نمایش تصویر رنگی
cv2.imshow('Original Image', image)

# نمایش تصویر خاکستری
cv2.imshow('Gray Image', gray_image)

# منتظر ماندن تا کلیدی فشار داده شود
cv2.waitKey(0)

# بستن پنجره ها
cv2.destroyAllWindows()
```

Command Prompt

```
C:\Users\Sarayetell>pip install opencv-python
```



تبدیل به مقیاس خاکستری

opencv.py > ...

```python
import cv2

# خواندن تصویر
image = cv2.imread('opencv.jpg')

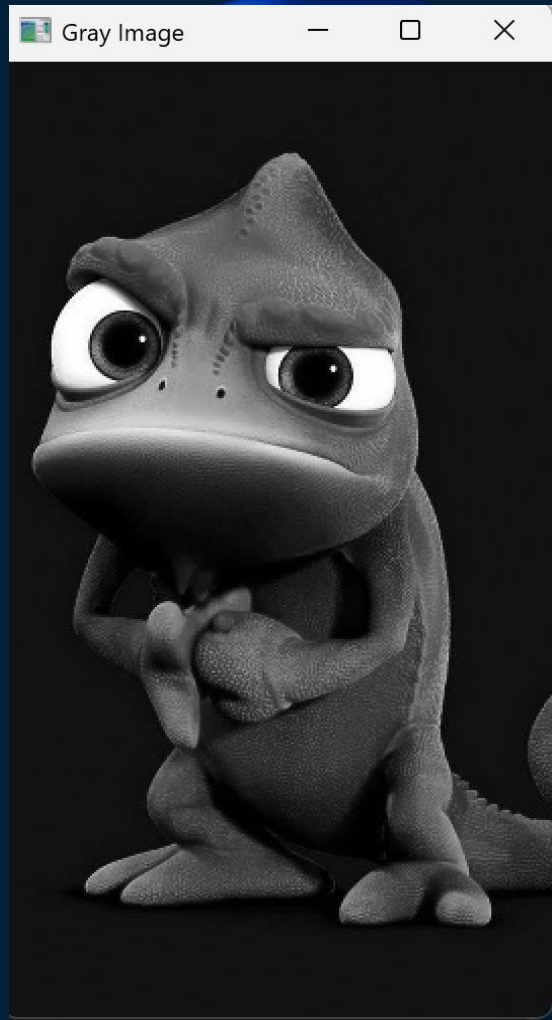
# تبدیل تصویر به مقیاس خاکستری
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# نمایش تصویر رنگی
cv2.imshow('Original Image', image)

# نمایش تصویر خاکستری
cv2.imshow('Gray Image', gray_image)

# منتظر ماندن تا کلیدی فشار داده شود
cv2.waitKey(0)

# بستن پنجره ها
cv2.destroyAllWindows()
```



تغییر اندازه تصویر

resize.py > ...

```python
import cv2
# بارگذاری تصویر
image = cv2.imread('Lizard.jpg')

# دریافت ابعاد اصلی تصویر
height, width = image.shape[:2]

# محاسبه نسبت جدید با حفظ نسبت ابعاد
new_width = 150
new_height = int((new_width / width) * height)

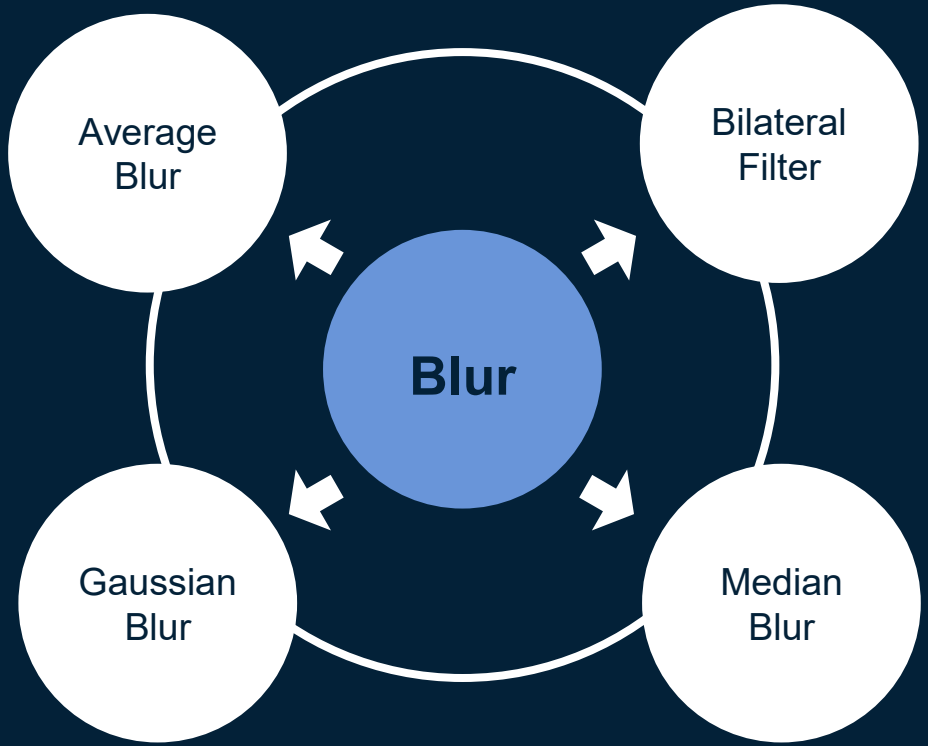
# تغییر اندازه تصویر
resized_image = cv2.resize(image, (new_width, new_height))

# نمایش تصویر
cv2.imshow('Resized Image', resized_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



نرم کردن تصویر





نرم کردن تصویر

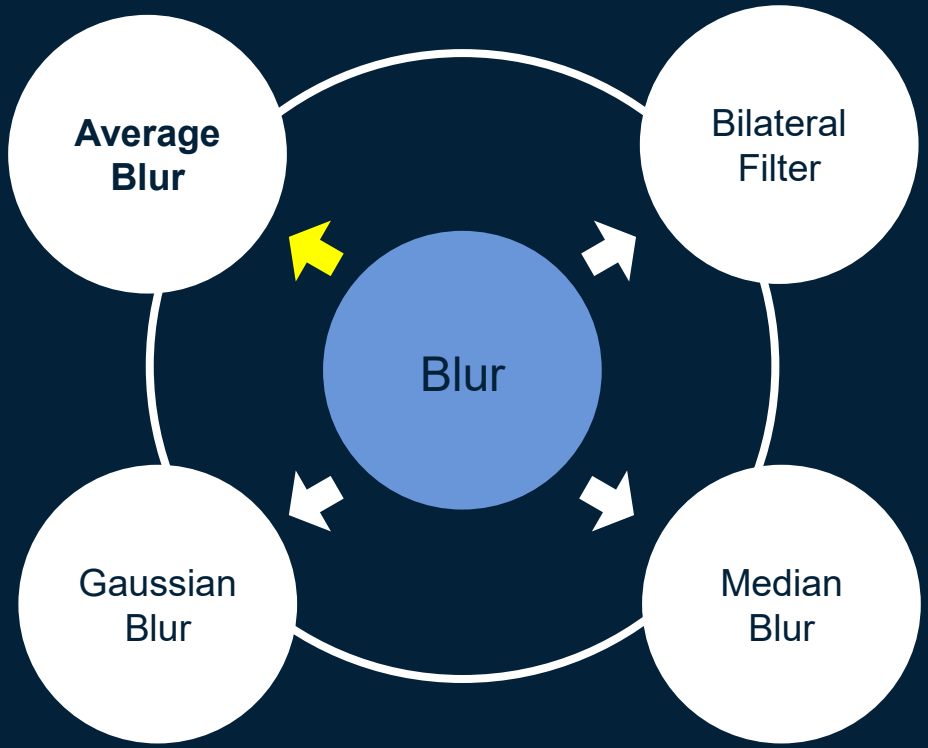
این روش با محاسبه میانگین پیکسل‌های اطراف هر پیکسل، تصویر را محو می‌کند.

ساده و سریع در پیاده سازی

مناسب برای کاهش نویز در تصاویر با کیفیت پایین

ممکن است لبه‌ها را به شدت محو کند و جزئیات تصویر را از بین ببرد.

زمانی که هدف فقط کاهش نویز است و حفظ جزئیات تصویر مهم نیست.



نرم کردن تصویر

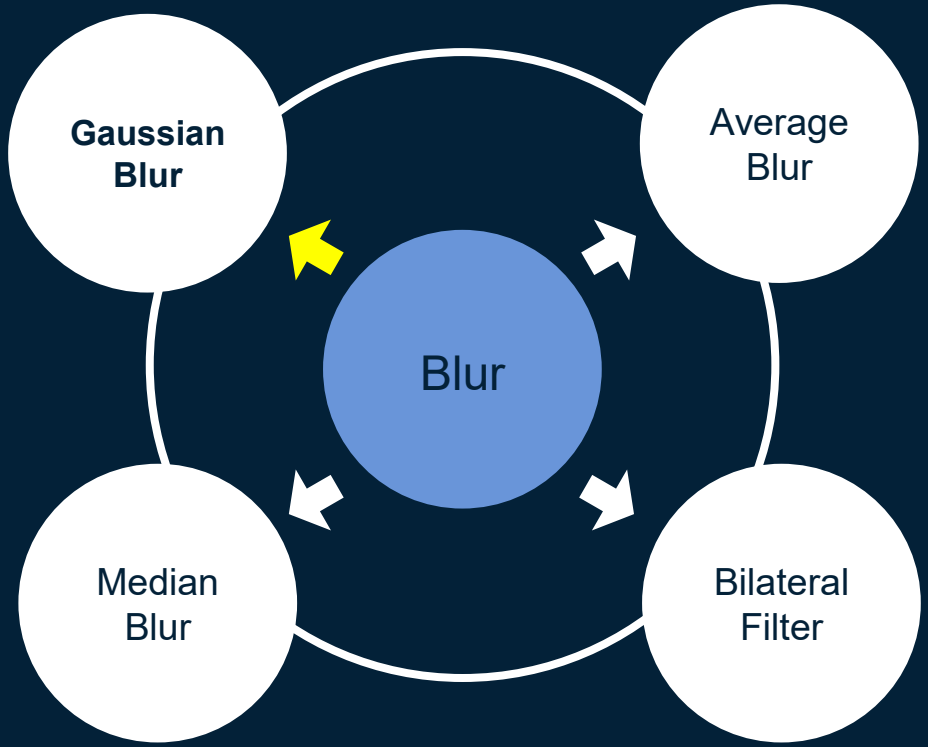
این روش از توزیع گاوسی برای محو کردن تصویر استفاده می‌کند و پیکسل‌های نزدیک‌تر به مرکز کرنل وزن بیشتری دارند.

نتایج بهتری نسبت به فیلتر میانگین

مناسب برای کاهش نویز و محو کردن جزئیات غیرضروری.

ممکن است لبه‌ها را به میزان کمتری از فیلتر میانگین محو کند، اما هنوز هم ممکن است جزئیات را تحت تأثیر قرار دهد.

در پردازش تصویر برای کاهش نویز و محو کردن جزئیات، به ویژه قبل از تشخیص لبه‌ها یا ویژگی‌ها



نرم کردن تصویر

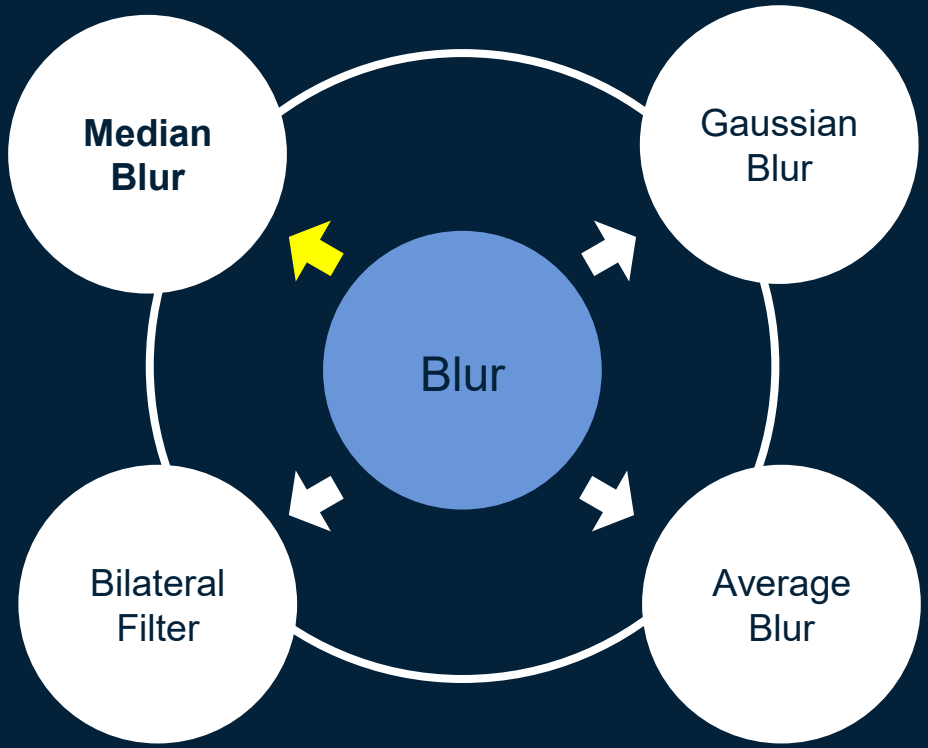
این روش با گرفتن میانه پیکسل‌های اطراف، تصویر را محو می‌کند.

بسیار مؤثر در حذف نویز نمک و فلفل بدون از بین بردن لبه‌ها.

حفظ جزئیات بهتر نسبت به فیلترهای میانگین و گاوسی

ممکن است زمان‌بر باشد، به ویژه برای تصاویر بزرگ

در شرایطی که تصویر دارای نویز نمک و فلفل است و نیاز به حفظ لبه‌ها و جزئیات وجود دارد.

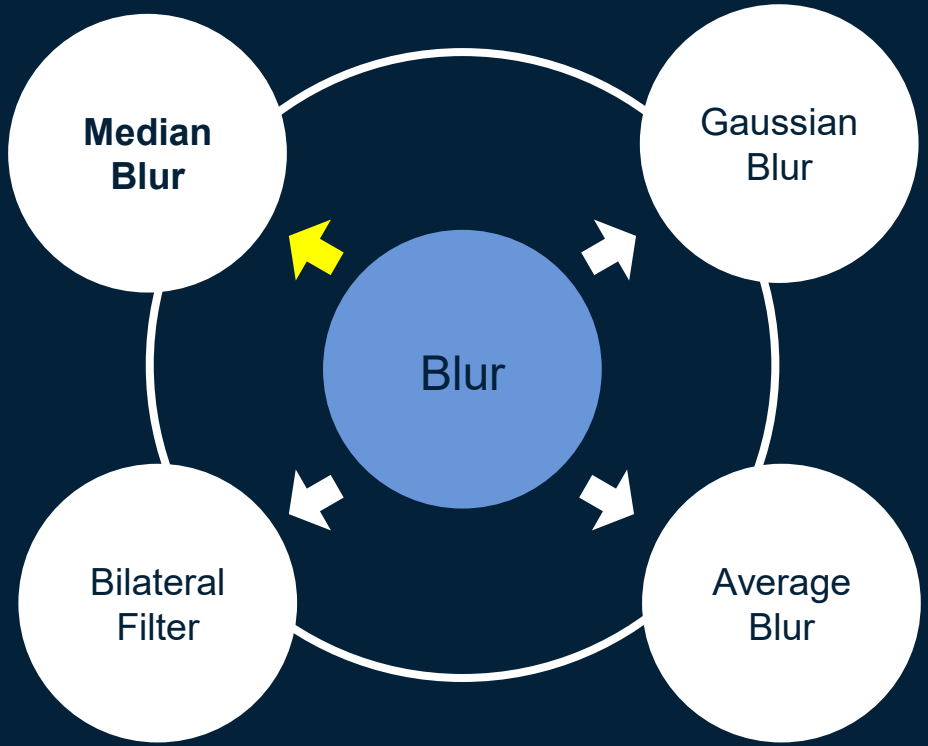


نرم کردن تصویر



Salt and pepper noise





نرم کردن تصویر

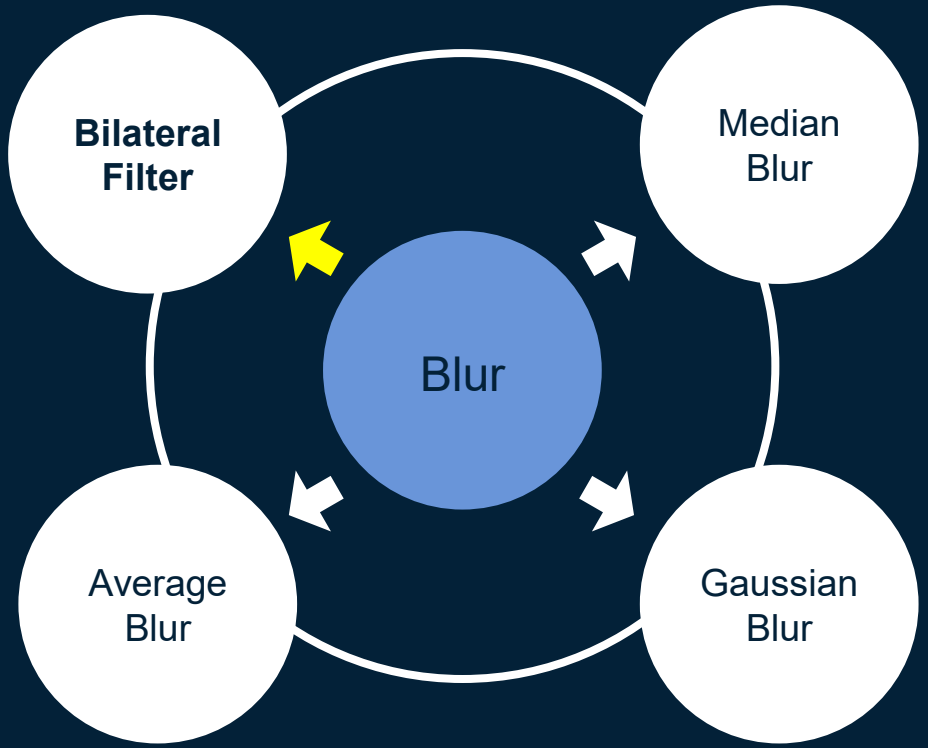
این فیلتر به‌طور خاص برای حفظ لبه‌ها در حالی که نویز را کاهش می‌دهد طراحی شده است. این کار با توجه به فاصله فضایی و اختلاف رنگ انجام می‌شود.

بهترین روش برای حفظ لبه‌ها و جزئیات در حالی که نویز را کاهش می‌دهد.

نتایج بسیار طبیعی و با کیفیت بالا

زمان‌برتر از سایر روش‌ها به دلیل محاسبات پیچیده‌تر و نیاز به تنظیم دقیق پارامترها

در پردازش تصویر پیشرفته، مانند آماده‌سازی تصاویر برای تشخیص اشیاء یا شناسایی ویژگی‌ها، جایی که حفظ جزئیات مهم است.



Bilateral Filter

opencv.py > ...

```python
import cv2

# بارگذاری تصویر
image = cv2.imread('Lizard.jpg')

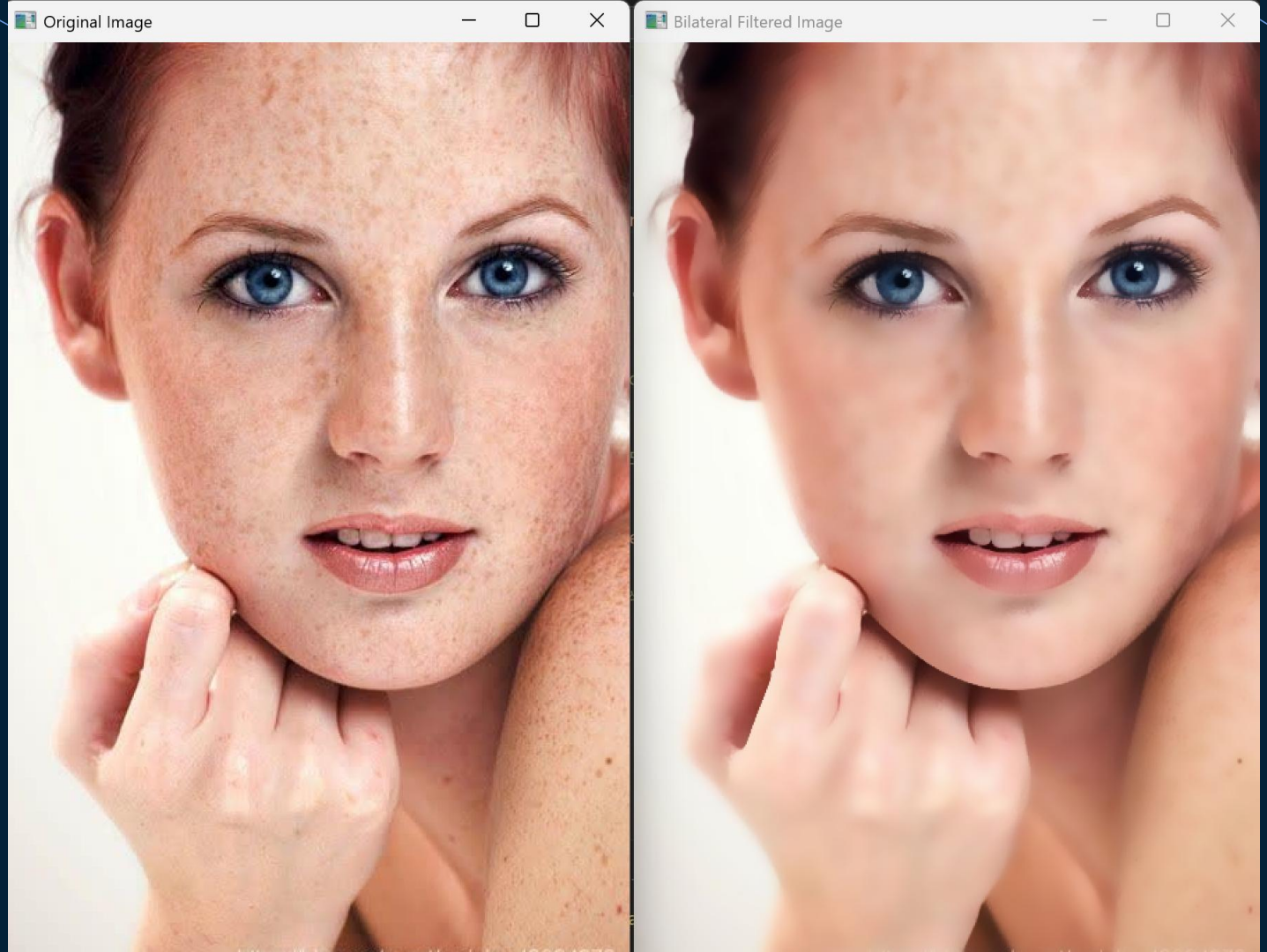
# نمایش تصویر اصلی
cv2.imshow('Original Image', image)

# اعمال Bilateral Filter
# src, kernel size, sigmaColor, sigmaSpace
bilateral_filtered_image = cv2.bilateralFilter(image, d=15, sigmaColor=75, sigmaSpace=75)

# d = ? (0, 1..5, 6..15, 15)

# نمایش تصویر فیلتر شده
cv2.imshow('Bilateral Filtered Image', bilateral_filtered_image)

# انتظار برای کلید ورودی و بستن پنجره‌ها
cv2.waitKey(0)
cv2.destroyAllWindows()
```


Original Image
Bilateral Filtered Image

تشخیص چهره با استفاده از Haar Cascades

opencv.py > ...

```python
import cv2

# بارگذاری تصویر
image = cv2.imread('himym.jpg')

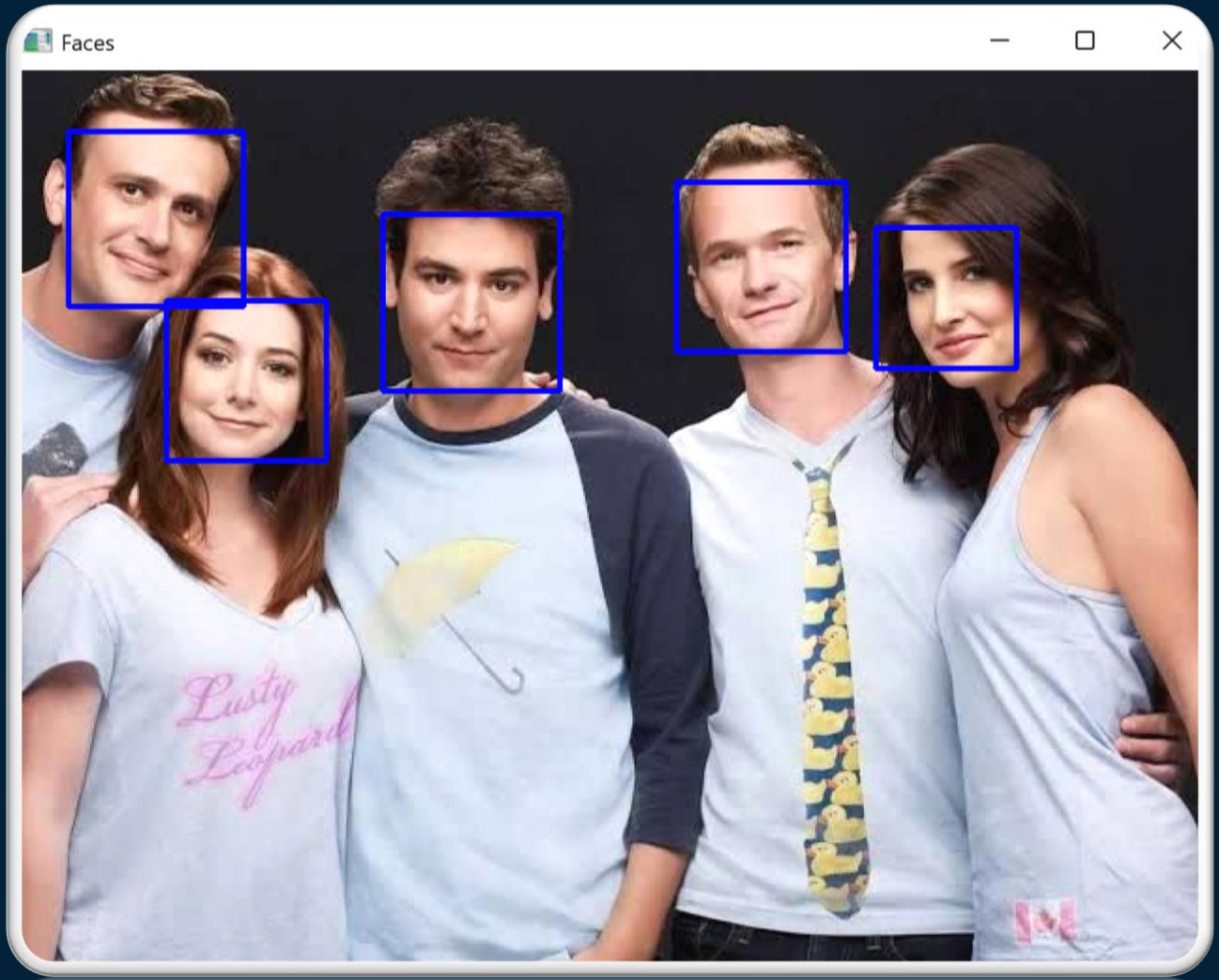
# بارگذاری مدل Haar Cascade
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')

# تبدیل تصویر به مقیاس خاکستری
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# تشخیص چهره ها
faces = face_cascade.detectMultiScale(gray_image, scaleFactor=1.1, minNeighbors=5)

# رسم مستطیل دور چهره ها
for (x, y, w, h) in faces:
    cv2.rectangle(image, (x, y), (x + w, y + h), (255, 0, 0), 2)

# نمایش تصویر
cv2.imshow('Faces', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```


Faces
Lusty Leopard

دانش واقعی در مسیر به اشتراک گذاشتن آن متولد می شود
سپاس از نگاه های مشتاق و ذهن های پرسشگر شما
☺☺☺☺☺☺

پرسش و پاسخ

